

UNIT 5 – GRAPHS

Graphs: Terminology used with Graph, Data Structure for Graph Representations: Adjacency Matrices, Adjacency List, Adjacency.

Graph Traversal: Depth First Search and Breadth First Search, Connected Component, Spanning Trees, Minimum Cost Spanning Trees: Prims and Kruskal algorithm. Transitive Closure and Shortest Path algorithm: Warshal Algorithm and Dijkstra Algorithm.

1. Introduction to Graphs

◇ Definition:

A **Graph** is a non-linear data structure consisting of a set of **vertices (nodes)** and **edges (connections)** between them.

👉 Formally:

A graph **G** is represented as

$$G = (V, E)$$

where:

- **V** → Set of vertices (nodes)
- **E** → Set of edges (connections between vertices)

◇ Example:

$V = \{A, B, C, D\}$
 $E = \{(A,B), (A,C), (B,D), (C,D)\}$

It means:

A is connected to B and C;

B and C are connected to D.

2. Terminology Used in Graphs

Term	Meaning	Example
Vertex (Node)	A point in the graph	A, B, C
Edge	Connection between two vertices	(A, B)
Degree of Vertex	Number of edges connected to a vertex	$\deg(A)=2$
Path	A sequence of vertices connected by edges	$A \rightarrow B \rightarrow C$

Cycle	Path that starts and ends at the same vertex	$A \rightarrow B \rightarrow C \rightarrow A$
Connected Graph	Every vertex is reachable from every other	All nodes connected
Disconnected Graph	Some nodes are not connected	Isolated nodes
Weighted Graph	Each edge has a weight/cost	$(A, B, 5)$
Directed Graph (Digraph)	Edges have direction	$A \rightarrow B$
Undirected Graph	Edges have no direction	$A - B$
Adjacent Vertices	Connected directly by an edge	A & B
Self-loop	Edge that connects a vertex to itself	(A, A)
Parallel Edges	Multiple edges between same vertices	Two edges from A to B

3. Graph Representations

There are **three main ways** to represent a graph in memory.

(a) Adjacency Matrix

A 2D array **adj[n][n]**, where each cell (i, j) is:

1 if there is an edge from $i \rightarrow j$

0 otherwise

Example:

For graph: $A \rightarrow B, A \rightarrow C, B \rightarrow D$

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

Pros: Easy to implement, quick to check edges.

Cons: Wastes space if graph is sparse.

(b) Adjacency List

Each vertex stores a list of connected vertices.

Example:

A → B → C B → D

C → D D → NULL

Pros: Saves memory for sparse graphs.

Cons: Slightly slower for checking if two vertices are directly connected.

(c) Adjacency Multi-list / Incidence Matrix

Used for **undirected graphs** with multiple edges between same vertices.

Rarely used in normal DSA syllabus but useful in network modeling.

4. Graph Traversal

Traversal = Visiting all the vertices systematically.

Two main traversal methods:

Depth First Search (DFS)

Breadth First Search (BFS)

◇ (a) Depth First Search (DFS)

Idea:

Go as deep as possible in one direction before backtracking.

Algorithm (using Stack / Recursion):

<pre>DFS(vertex v) mark v as visited for every adjacent vertex u of v: if u not visited: DFS(u)</pre>

Example:

Graph edges: A-B, A-C, B-D, C-E

Traversal order (starting from A):

👉 A → B → D → C → E

Time Complexity: $O(V + E)$

◇ (b) Breadth First Search (BFS)

Idea:

Explore all neighbors of a node first before moving deeper (level by level).

Algorithm (using Queue):

```
BFS(vertex v)
  create a queue Q
  mark v as visited
  enqueue(v)
  while Q not empty:
    u = dequeue(Q)
    for each adjacent vertex w of u:
      if w not visited:
        mark w as visited
        enqueue(w)
```

Example:

Graph edges: A-B, A-C, B-D, C-E

Traversal order (starting from A):

👉 A → B → C → D → E

Time Complexity: $O(V + E)$

5. Connected Components

A **connected component** is a subgraph in which every pair of vertices is connected, and no other vertices are connected outside it.

Example:

If graph has edges:

(A-B-C) and (D-E)

→ There are **two connected components**: {A,B,C} and {D,E}.

6. Spanning Trees

◇ **Definition:**

A **spanning tree** of a graph is a subgraph that:

Includes **all vertices**

Is **connected**

Has **no cycles**

If a graph has **V vertices**, the spanning tree has **(V - 1) edges**.

Example:

For graph with 4 vertices, spanning tree = 3 edges connecting all nodes.

7. Minimum Cost Spanning Tree (MST)

A spanning tree where the **sum of edge weights is minimum**.

Two main algorithms:

Prim's Algorithm

Kruskal's Algorithm

◇ (a) Prim's Algorithm

Idea:

Grow the MST by adding the **cheapest edge** that connects a new vertex to the tree.

Steps:

Start from any vertex.
Choose the smallest edge that connects the tree to a new vertex.
Repeat until all vertices are included.

Complexity: $O(V^2)$

Used for: Dense graphs.

◇ (b) Kruskal's Algorithm

Idea:

Sort all edges by weight and keep adding the smallest edge **that doesn't form a cycle**.

Steps:

Sort edges in increasing weight order.
Add edges one by one, skipping those that form a cycle (using Union-Find).
Stop when $(V - 1)$ edges are chosen.

Complexity: $O(E \log E)$

Used for: Sparse graphs.

8. Transitive Closure

Transitive closure of a graph tells **which vertices are reachable** from others.

Warshall's Algorithm is used for this.

◇ Warshall's Algorithm

Idea:

It modifies the adjacency matrix to show reachability.

Algorithm:

```
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      path[i][j] = path[i][j] OR (path[i][k] AND path[k][j])
```

Output:

A matrix showing 1 if there's a path from i to j.

Complexity: $O(V^3)$

9. Shortest Path Algorithms

♦ Dijkstra's Algorithm (Single Source Shortest Path)

Used for: Finding shortest distance from one source vertex to all other vertices (no negative edges).

Steps:

Assign distance = 0 to source, ∞ to all others.
Select the vertex with minimum distance (not yet visited).
Update distances of adjacent vertices.
Repeat until all vertices are visited.

Complexity: $O(V^2)$

♦ Example

Graph edges (weights):

A-B: 4
A-C: 2
B-C: 1
B-D: 5
C-D: 8

Shortest distances from A:

Vertex	Distance
A	0
B	3
C	2
D	8

10. Complexity Summary Table

Operation	Algorithm	Time Complexity
Graph Traversal	BFS / DFS	$O(V + E)$
Spanning Tree	Prim's	$O(V^2)$
	Kruskal's	$O(E \log E)$
Transitive Closure	Warshall's	$O(V^3)$
Shortest Path	Dijkstra's	$O(V^2)$

Important Difference Tables – Unit 5:

Graphs

1. Directed Graph vs Undirected Graph

Feature	Directed Graph (Digraph)	Undirected Graph
Definition	Edges have a direction ($A \rightarrow B$).	Edges have no direction ($A - B$).
Edge Representation	Ordered pair (u, v) .	Unordered pair $\{u, v\}$.
Example	Twitter followers (A follows B).	Facebook friendship (A and B are friends).
Degree of Vertex	Has In-degree & Out-degree .	Has a single degree (no direction).
Adjacency Matrix	Not symmetric.	Symmetric matrix.
Traversal	Must follow direction of edges.	Can traverse both ways.

2. Weighted Graph vs Unweighted Graph

Feature	Weighted Graph	Unweighted Graph
Definition	Each edge has a weight (cost, distance, time).	All edges are equal; no weights.
Edge Representation	$(A, B, 5) \rightarrow \text{cost} = 5$.	(A, B) only.
Example	Road map with distances.	Simple connection graph.
Used In	Dijkstra's, Kruskal's, Prim's.	BFS, DFS, connectivity check.

3. Adjacency Matrix vs Adjacency List

Feature	Adjacency Matrix	Adjacency List
Storage	2D array of size $V \times V$.	Array of linked lists.
Space Required	$O(V^2)$.	$O(V + E)$.
Best For	Dense graphs (many edges).	Sparse graphs (few edges).
Edge Check	Fast ($O(1)$).	Slower ($O(V)$).
Add/Delete Edge	Easy to update values.	Easier to add/remove dynamically.
Example Use	Warshall's algorithm.	BFS, DFS implementations.

4. DFS vs BFS (Depth First Search vs Breadth First Search)

Feature	DFS	BFS
---------	-----	-----

Approach	Goes deep before backtracking.	Explores all neighbors level by level.
Data Structure Used	Stack (or recursion).	Queue.
Traversal Order Example	$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$.	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$.
Memory Usage	Less for wide graphs.	More (stores all neighbors).
Applications	Topological sorting, cycle detection, maze solving.	Shortest path (unweighted graphs), connectivity.
Complexity	$O(V + E)$.	$O(V + E)$.

5. Prim's Algorithm vs Kruskal's Algorithm

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Grows MST by adding the nearest vertex.	Adds smallest edges without forming a cycle.
Starting Point	Starts from a single vertex.	Works on all edges directly.
Data Structure Used	Adjacency matrix, priority queue.	Edge list + Union-Find (Disjoint Set).
Graph Type Best For	Dense graphs.	Sparse graphs.
Cycle Check	Automatically avoids cycles.	Must check using Union-Find.
Complexity	$O(V^2)$ (simple version).	$O(E \log E)$.
Example	Network cabling from a single node.	Connecting different clusters.

6. Warshall's Algorithm vs Dijkstra's Algorithm

Feature	Warshall's Algorithm	Dijkstra's Algorithm
Purpose	Finds <i>reachability</i> (transitive closure).	Finds <i>shortest path</i> from one source.
Input	Adjacency matrix (0/1 edges).	Weighted graph (positive weights).
Output	Matrix showing reachable vertices.	Shortest distance from source to all vertices.
Type of Graph	Directed/Unweighted.	Weighted (no negative edges).
Complexity	$O(V^3)$.	$O(V^2)$.
Example Use	Checking if a path exists between two cities.	Finding minimum travel cost.

7. Connected Graph vs Disconnected Graph

Feature	Connected Graph	Disconnected Graph
---------	-----------------	--------------------

Definition	Every vertex can be reached from any other vertex.	At least one vertex pair is not connected.
Number of Components	Only one connected component.	Two or more components.
Example	A—B—C—D (single chain).	(A—B—C) and (D—E).
Traversal Output	Visits all vertices in one go.	Each component traversed separately.

? 8. Spanning Tree vs Minimum Spanning Tree (MST)

Feature	Spanning Tree	Minimum Spanning Tree
Definition	Connects all vertices with $(V-1)$ edges, no cycles.	A spanning tree with <i>minimum total edge weight</i> .
Weight Consideration	Not considered.	Sum of weights must be minimum.
Number of Possible Trees	Can be many.	Only one or few with minimum cost.
Algorithms	DFS, BFS (to form tree).	Kruskal's, Prim's.
Example	Any connected tree.	Least-cost network of roads.

9. Sparse Graph vs Dense Graph

Feature	Sparse Graph	Dense Graph
Number of Edges	Few edges compared to max possible.	Close to maximum possible edges.
Storage Preferred	Adjacency list.	Adjacency matrix.
Time Complexity (Traversals)	Faster for large V .	Slower due to $O(V^2)$ space.
Example	Road map of rural area.	Airline network (many routes).

? 10. Single Source vs All-Pairs Shortest Path

Feature	Single Source Shortest Path	All-Pairs Shortest Path
Purpose	Shortest path from one vertex to all others.	Shortest paths between every pair of vertices.
Algorithm Used	Dijkstra's / Bellman-Ford.	Floyd-Warshall / Warshall's.
Output Type	One array of distances.	A complete matrix.
Complexity	$O(V^2)$	$O(V^3)$
Use Case	GPS from one location.	Routing tables in networks.

Notes by - [GKM](#)